

AlphaTrade India — API ↔ UI Mapping Document (v1.0)

Purpose: This document defines the **exact contract between Backend APIs and Frontend UI components**.

It answers one critical question:

Which backend endpoint feeds which UI component, how often, and by which transport (REST / WebSocket)?

This is a **powerful control document** that prevents:

- Over-fetching
 - Tight coupling
 - UI confusion
 - Backend breaking frontend
-

1. Data Transport Strategy (Global)

Transport Rules

Data Type	Transport	Reason
Live prices	WebSocket	Low latency
P&L / MTM	WebSocket	Real-time
Positions	WebSocket + REST fallback	Sync safety
Config / Settings	REST	Persistence
Actions (Kill, Save)	REST (POST)	Deterministic
Logs / History	REST	Paginated

2. Authentication & Session APIs

Backend Endpoints

Endpoint	Method	Purpose
/auth/login	POST	Broker login
/auth/status	GET	Session validity
/auth/logout	POST	Logout / reset

UI Mapping

UI Page	Component	API	Trigger
Login	LoginForm	/auth/login	On submit
App Shell	ConnectionBadg e	/auth/statu s	App load + interval

3. Dashboard — Live Metrics Mapping

WebSocket Channel: **/ws/metrics**

Data Field	UI Component	Update Frequency
net_pnl	NetPnlCard	Real-time
margin_used	MarginUsedCard	Real-time
available_balanc e	BalanceCard	Real-time
trades_count	TradesCountCar d	On trade

4. Live Market Watch Mapping

WebSocket Channel: **/ws/market**

Data Field	UI Component	Description
------------	--------------	-------------

symbol	MarketRow	Stock identifier
ltp	PriceTicker	Last traded price
vwap	VWAPIndicator	VWAP value
above_vwap	VWAPIndicator	Boolean state

5. Positions & Trades Mapping

WebSocket Channel: **/ws/positions**

Data Field	UI Component	Purpose
open_positions[]	PositionsTable	Active trades
qty	PositionsRow	Quantity
entry_price	PositionsRow	Entry
unrealized_pnl	PnLBadge	Live P&L

REST Fallback

Endpoint	Method	Component
/positions/open	GET	PositionsTable
/trades/today	GET	TradesTable

6. Risk & Control Mapping

REST Endpoints

Endpoint	Method	Purpose
/risk/status	GET	Risk state

/risk/kill	POST	Emergency exit
/risk/lock	POST	Trading lock

UI Mapping

Component	API	Trigger
RiskMeter	/risk/status	Poll / WebSocket
KillSwitchButton	/risk/kill	User click
TradingStatusBadge	/risk/status	On update

7. Stock Selection Mapping

REST Endpoints

Endpoint	Method	Purpose
/stocks/universe	GET	Search universe
/stocks/selected	GET	Current selection
/stocks/select	POST	Save selection
/stocks/scan	GET	Auto scanner

UI Mapping

Component	API	Trigger
StockSearchInput	/stocks/universe	On type
SelectedStockChip	/stocks/selected	Page load
ScannerResultTable	/stocks/scan	Manual trigger

ApplySelectionButton /stocks/select Save

8. Settings Page Mapping

REST Endpoints

Endpoint	Method	Purpose
/config/get	GET	Load config
/config/save	POST	Save config

UI Mapping

Component	API	Trigger
CapitalInput	/config/get	Page load
MaxTradesDropdown	/config/get	Page load
SaveSettingsButton	/config/save	Save click
SettingsSummaryCard	/config/get	Refresh

9. Logs & History Mapping

REST Endpoints

Endpoint	Method	Purpose
/logs/system	GET	Engine logs
/logs/trades	GET	Trade history

UI Mapping

Component	API	Trigger
TradeHistoryTable	/logs/trades	Page load

LogTable	/logs/system	Page load
DateFilter	/logs/*	Filter change
ExportButton	/logs/trades	Export

10. System Status Mapping

REST Endpoints

Endpoint	Method	Purpose
/system/health	GET	Overall health
/system/broker	GET	Broker status
/system/ws	GET	WebSocket status

UI Mapping

Component	API	Trigger
StatusCard	/system/health	Page load
LatencyMetric	/system/health	Interval
WebSocketStatus	/system/ws	Page load
RefreshStatusButton	/system/*	Manual refresh

11. Error & Empty State Mapping

Error Sources

Error Type	UI Response
------------	-------------

API 401	Force logout
API 500	Error banner
WS disconnect	Red status badge
Broker reject	Toast + log

12. API Contract Rules (Non-Negotiable)

1. No breaking response shapes
 2. Version APIs if changed
 3. Backend never sends UI logic
 4. UI never calculates financial truth
 5. WebSocket messages must be idempotent
-

13. Frontend Development Workflow

1. UI component defined
 2. Data contract mapped here
 3. Mock API created
 4. UI built against contract
 5. Backend implements endpoint
-

14. Final Statement

This document is the **spine of frontend-backend collaboration**.

If this stays correct:

- Frontend remains stable
- Backend evolves safely
- System scales cleanly

APIs serve the UI.

UI never guesses data.